How do we measure one account's return and risk?

First calculate gains and declines over the actual model period, then examine the limits of scaling, resampling and retrospective selection.

Issue	What to evaluate this week
01 to 03	Daily returns, compounding and annualization
04 to 09	Volatility, targets, Sharpe and Sortino
10 to 14	Drawdown, recovery and different periods
15 to 17	Adjacent returns and covariance
18 to 21	Block resampling and sensitivity to settings
22 to 24	Rule selection and complete-account checks

Use Week 6 prices, trades and costs. Changing evaluation measures does not add account income. This week does not select portfolio weights.

AAPL account and daily returns

2024/1/2 to 6/28; initial wealth USD 1,000. A positive one-day price return requests a stock position, executed at the next trading-day close.

Allow fractional shares; exclude dividends, taxes and cash interest. Approximate each purchase or sale cost as 4bp of that interval's opening account wealth. Only value holdings on the final day; do not trade.

Before running: If only evaluation is added, should the 6/28 account still equal Week 6's USD 1109.43?

```
import numpy as np
import pandas as pd
import yfinance as yf
aapl = yf.Ticker("AAPL").history(
    start="2024-01-02", end="2024-07-01", interval="1d",
    auto_adjust=False, actions=True)
aapl.index = aapl.index.strftime("%Y-%m-%d")
close = aapl["Close"]
asset_return = close.pct_change(fill_method=None)
signal = asset_return.gt(0).astype(float).where(asset_return.notna())
held = signal.shift(2).fillna(0)  # Interval stock weight: 1 stock, 0 cash
after = signal.shift(1).fillna(0)  # Stock weight after this closing trade
after.iloc[-1] = held.iloc[-1]
turnover = (after-held).abs()  # Absolute weight change, not shares
gross = asset_return.where(held.eq(1), 0.0)  # Account return before costs, in decimals
net = gross-.0004*turnover  # Account return after costs, in decimals
wealth = 1000*(1+net).cumprod(skipna=False)  # Total account wealth in USD
r = net.iloc[1:]  # 123 net returns, excluding initial 1/2
n = len(r)  # Number of return intervals
dates = ["2024-01-09", "2024-01-10", "2024-06-28"]
pd.DataFrame({"Observation date": dates,
    "Close (USD/share)": close.loc[dates].round(2).values,
    "Account (USD)": wealth.loc[dates].round(2).values})
```

Observation date	Close (USD/share)	Account (USD)
2024-01-09	185.14	999.6
2024-01-10	186.19	1004.87
2024-06-28	210.62	1109.43

124 closes form 123 intervals; 1/2 is initial wealth of USD 1,000, not another daily return. For 1/9, held = 0 (cash) and after = 1 following the 1/9 closing trade. For 1/10, held = 1 and after = 0 following the closing sale. Both turnover values are 1: stock weight changes 0→1 or 1→0, not one share. Terminal wealth remains USD 1109.43.

ISSUE 01

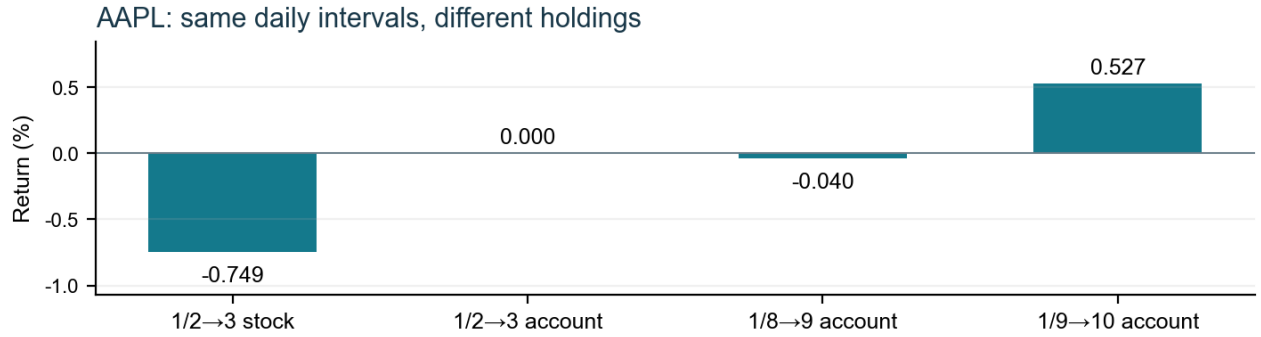
Must the account lose when the stock falls?

Identify what is held during the interval, then include the day's trading cost. Cash days waiting for purchase also count as intervals.

Before running: With cash throughout 1/2 to 3 and no trade, does a stock decline reduce the account?

```
dates = ["2024-01-03", "2024-01-09", "2024-01-10"]
pd.DataFrame({"Interval end": dates,
    "Stock (%)": (100*asset_return.loc[dates]).round(4).values,
    "Account net (%)": (100*r.loc[dates]).round(4).values})
```

Interval end	Stock (%)	Account net (%)
2024-01-03	-0.7488	0.0
2024-01-09	-0.2263	-0.04
2024-01-10	0.5671	0.5271



The account returns 0% on 1/3. Purchase cost gives -0.04% on 1/9; the stock gain less sale cost gives +0.5271% on 1/10.

Both 1/3 and 1/9 intervals hold cash (held = 0), so stock moves do not enter the account. Buying at the 1/9 close changes weight 0→1 and only deducts 0.04%. The 1/10 interval has held = 1 and earns the stock gain; the closing sale changes weight 1→0 and deducts another 0.04%. Main statistics retain all 123 intervals: cash, stock and cost-paying days, not only stock-held dates.

Discussion

Which account expense on 1/9 would be omitted by retaining only stock-held days?

Sources: [1], [5].

ISSUE 02

Which average reproduces USD 1109.43?

The arithmetic mean describes an average interval return. The geometric mean finds a constant interval growth rate whose 123-interval compounding matches this account.

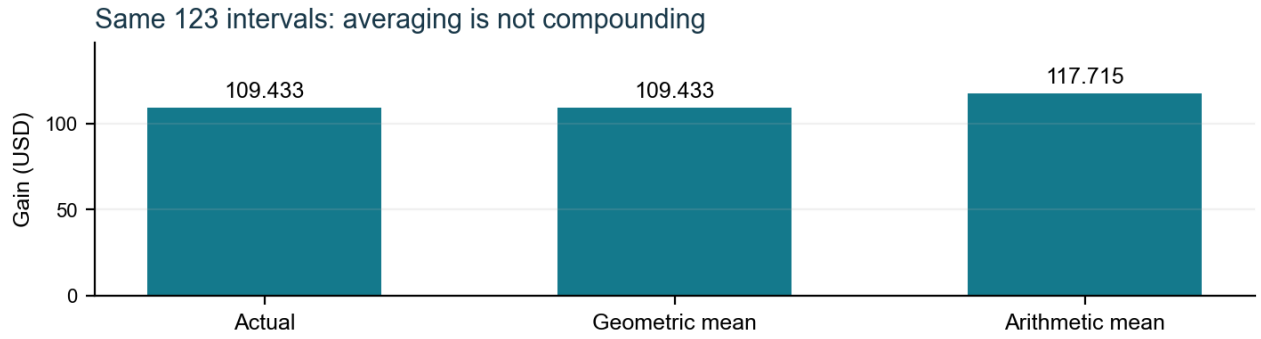
r is a decimal daily net return, with a bar denoting its mean; g is a constant daily growth rate. Initial wealth is USD 1000. Average log(1+r), exponentiate for the growth factor, then subtract 1 for g. This equals multiplying the 123 growth factors, taking the 123rd root, then subtracting 1.

$$\bar{r} = \frac{\sum r}{123} \approx 0.000905176, \quad (1+g)^{123} = \frac{1109.433331}{1000}$$

Before running: Which constant daily growth rate repeated 123 times matches the actual account?

```
mean_return = r.mean()
geometric = np.expm1(np.log1p(r).mean())
pd.DataFrame({"Method": ["Actual account", "Repeat geometric mean", "Repeat arithmetic mean"],
    "Terminal (USD)": [wealth.iloc[-1], 1000*(1+geometric)**n,
                       1000*(1+mean_return)**n]}).round(2)
```

Method	Terminal (USD)
Actual account	1109.43
Repeat geometric mean	1109.43
Repeat arithmetic mean	1117.71



The daily geometric mean 0.0844660% reproduces the USD 109.43 gain. Repeating the daily arithmetic mean 0.0905176% does not.

Discussion

Does 123 times the arithmetic mean, about 11.1337%, equal the account gain of 10.9433%?

Sources: [2], [5].

ISSUE 03

Does annualized 23.709% mean a full year's gain was earned?

Scaling growth from 123 trading intervals to 252 intervals does not add observed trades.

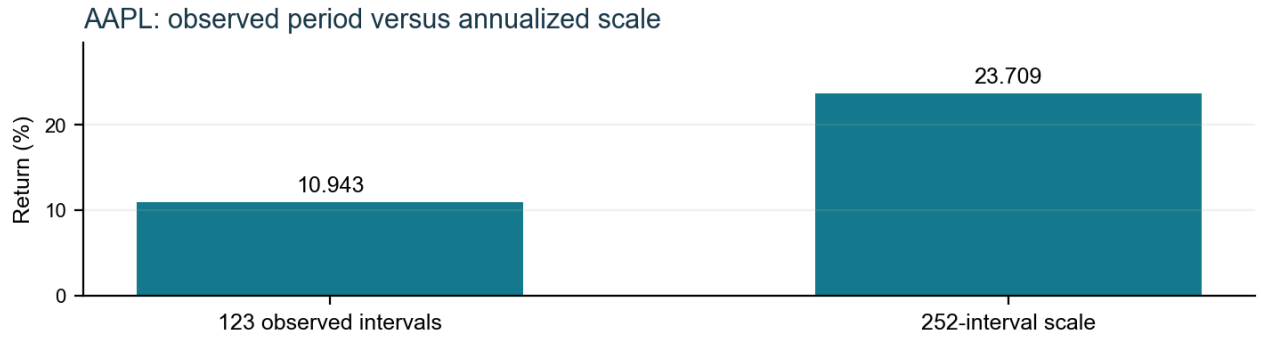
R is the teaching annualized rate. The common convention of 252 trading days per year sets a scale, not this sample's size. Solve for a constant annual rate R giving growth factor 1109.433331/1000 over 123/252 of a year.

$$(1+R)^{123/252} = \frac{1109.433331}{1000} \quad \Rightarrow \quad R = \left(\frac{1109.433331}{1000}\right)^{252/123} - 1$$

Before running: Positive growth becomes a larger number on a longer scale, but has market data been added?

```
observed_gain = wealth.iloc[-1]/1000-1
annualized_return = (1+observed_gain)**(252/n)-1
pd.DataFrame({"Period or scale": ["1/2 to 6/28: 123 intervals", "Teaching scale: 252 intervals"],
    "Return (%)": [100*observed_gain, 100*annualized_return]}).round(3)
```

Period or scale	Return (%)
1/2 to 6/28: 123 intervals	10.943
Teaching scale: 252 intervals	23.709



10.943% is this model-period gain; 23.709% is a scaling calculation, not a realized full-year gain or a next-year forecast.

GIPS does not permit presenting annualized returns for periods shorter than one year. This page demonstrates scaling, not a compliant performance report.

Discussion

Which number describes the model gain already realized over 1/2 to 6/28 to an investor?

Sources: [5].

ISSUE 04

How do daily deviations combine into volatility?

Sample standard deviation squares deviations before averaging and taking the square root, preventing positive and negative deviations from directly canceling.

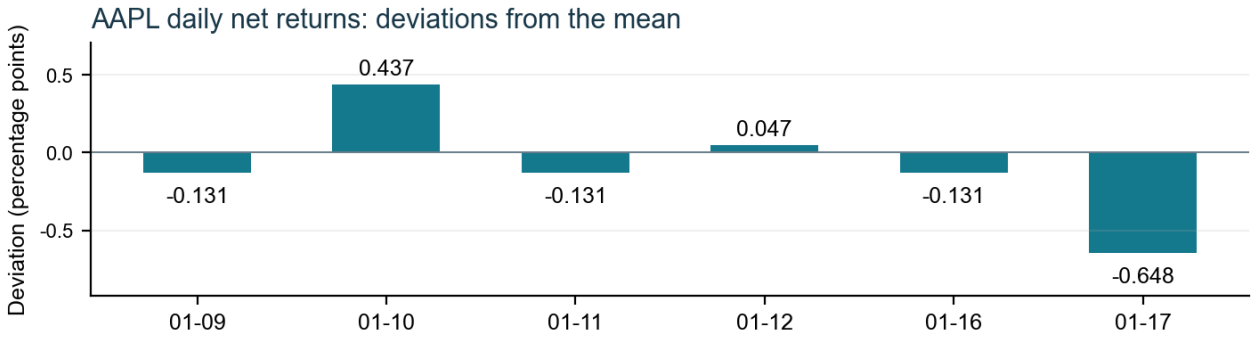
r is a decimal daily net return with mean about 0.000905176. s is the sample standard deviation of 123 observations, using denominator n-1 = 122.

$$s = \sqrt{\frac{\sum (r - \bar{r})^2}{122}} \approx \sqrt{\frac{0.0151154063}{122}} \approx 0.01113089$$

Before running: -0.04% is below the +0.0905176% mean. Will its squared deviation retain a negative sign?

```
daily_sd = r.std(ddof=1)
dates = ["2024-01-09", "2024-01-10", "2024-01-17"]
deviation = 100*(r.loc[dates]-mean_return)
pd.DataFrame({"Observation date": dates,
    "Net return (%)": (100*r.loc[dates]).round(4).values,
    "Deviation (percentage points)": deviation.round(4).values,
    "Squared (percentage points²)": deviation.pow(2).round(4).values})
```

Observation date	Net return (%)	Deviation (percentage points)	Squared (percentage points²)
2024-01-09	-0.04	-0.1305	0.017
2024-01-10	0.5271	0.4366	0.1906
2024-01-17	-0.5574	-0.6479	0.4197



The 1/9 deviation is about -0.1305 percentage points. Sum squares for all 123 observations, divide by 122 and take the square root for daily SD of 1.1131%.

Discussion

Why can't adding negative and positive deviations replace standard deviation?

Sources: [2], [5].

ISSUE 05

Why multiply daily volatility by √252, not 252?

With equal variances and zero covariance across different intervals, variances add; standard deviation then takes the square root.

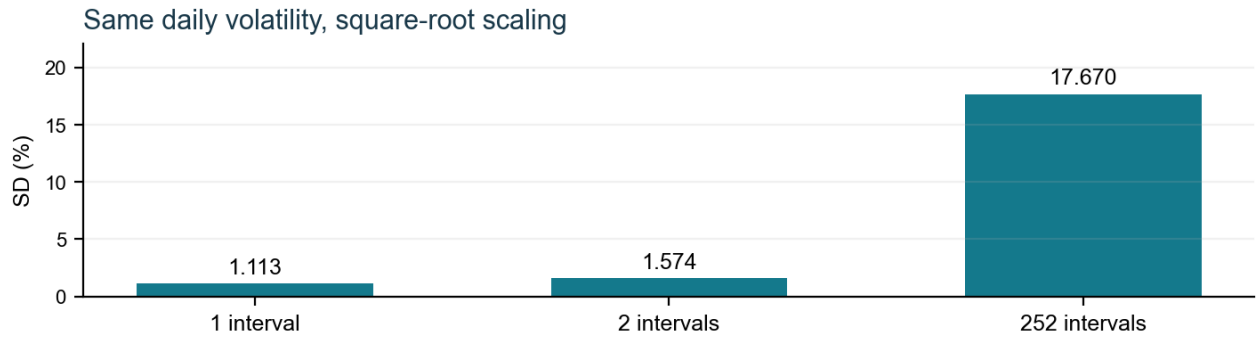
Covariance measures whether two returns deviate from their own means in the same (positive) or opposite (negative) direction. σ is theoretical daily SD. Assume it is constant each day and covariance is zero for every pair of distinct trading days. Add 252 interval variances, then take the square root. Estimate σ using Issue 4's net daily sample SD, s=1.113089%.

$$\sqrt{252\sigma^2} = \sigma\sqrt{252}, \qquad s\sqrt{252} \approx 1.113089\%\sqrt{252} \approx 17.6697\%$$

Before running: Is the 252-interval scale about 15.87 or 252 times daily SD?

```
annual_sd = daily_sd*np.sqrt(252)
pd.DataFrame({"Intervals summed": [1, 2, 252],
    "Zero-covariance scaling (%)": [100*daily_sd,
        100*daily_sd*np.sqrt(2), 100*annual_sd]}).round(4)
```

Intervals summed	Zero-covariance scaling (%)
1	1.1131
2	1.5741
252	17.6697



17.6697% is conditional scaling, not the exact volatility of annual compounded returns or a maximum loss.

Scaling proves neither daily independence nor equal volatility. Later checks cover only adjacent days, not all trading-day pairs.

Discussion

If adjacent returns often deviate in the same direction, what would summing daily variances alone omit?

Sources: [2], [3], [5].

ISSUE 06

What daily reference corresponds to 5% per year?

Daily strategy returns require a daily reference. The effective annual rate of 5% is assumed here, not an observed interest rate.

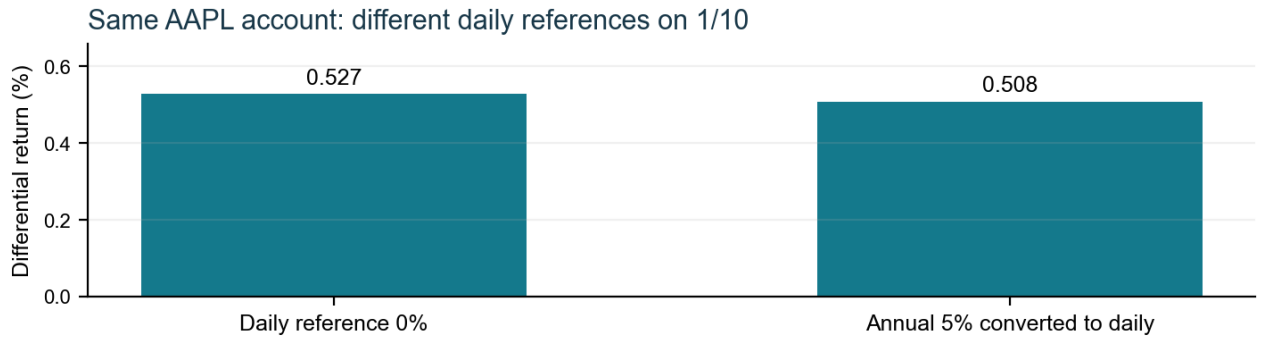
r_f is a constant daily reference rate. With 252 intervals per year, multiplying its growth factor 252 times must give 1.05. The main account and later main ratios still use a daily reference of 0%.

$$(1 + r_f)^{252} = 1.05 \quad \Rightarrow \quad r_f = 1.05^{1/252} - 1 \approx 0.019363\%$$

Before running: After converting 5% to a daily rate, what should compounding 252 times recover?

```
annual_targets = np.array([0., .05])
daily_targets = (1+annual_targets)**(1/252)-1
pd.DataFrame({"Assumed annual rate (%)": 100*annual_targets,
    "Daily reference (bp)": (10000*daily_targets).round(4),
    "Compounded annual rate (%)": (100*((1+daily_targets)**252-1)).round(4)})
```

Assumed annual rate (%)	Daily reference (bp)	Compounded annual rate (%)
0.0	0.0	0.0
5.0	1.9363	5.0



The 1/10 net return is 0.5271%. Subtract daily 0.019363% for about 0.5078%, not the annual 5%.

Discussion

What is wrong with subtracting 5% each day?

Sources: [2], [4].

ISSUE 07

How large is mean return relative to volatility?

The Sharpe ratio divides mean differential return by its standard deviation at the same frequency, not total gain by maximum drawdown.

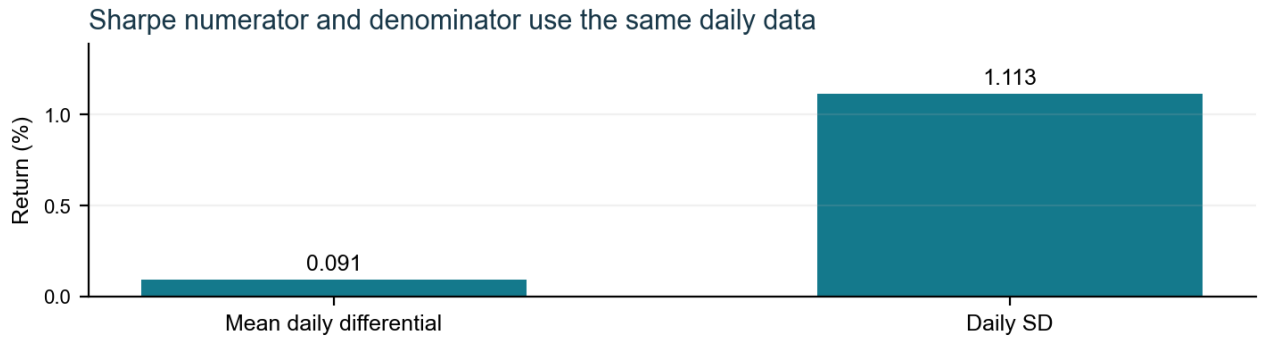
SR denotes Sharpe. With daily reference zero, use the 123-interval mean 0.0905176% in the numerator and the same 123 intervals' SD 1.1130893% in the denominator.

$$SR_{\text{daily}} = \frac{0.0905176\%}{1.1130893\%} \approx 0.0813211, \quad SR_{\text{annual}} = SR_{\text{daily}}\sqrt{252}$$

Before running: Can a positive mean divided by a positive SD give a negative ratio?

```
reference = 0.0
differential = r-reference
sharpe_daily = differential.mean()/differential.std(ddof=1)
sharpe_annual = sharpe_daily*np.sqrt(252)
pd.DataFrame({"Scale": ["Daily", "Conventional annualization"],
    "Sharpe ratio": [sharpe_daily, sharpe_annual]}).round(6)
```

Scale	Sharpe ratio
Daily	0.081321
Conventional annualization	1.290932



0.0905 divided by 1.1131 gives daily 0.081321. Multiply by √252 for 1.290932, not an additional 129% gain.

The annualized numerator is the daily arithmetic mean times 252, not Issue 3's compounded annualized rate. Cash has zero interest and volatility, so Sharpe is undefined, not automatically 0.

Discussion

Does dividing compounded annualized 23.709% by annualized volatility 17.6697% give this page's Sharpe?

Sources: [2], [3].

ISSUE 08

How do we measure risk below a target?

Downside deviation sets target-meeting returns to zero, then squares shortfalls, averages and takes the square root. The target here is daily 0%.

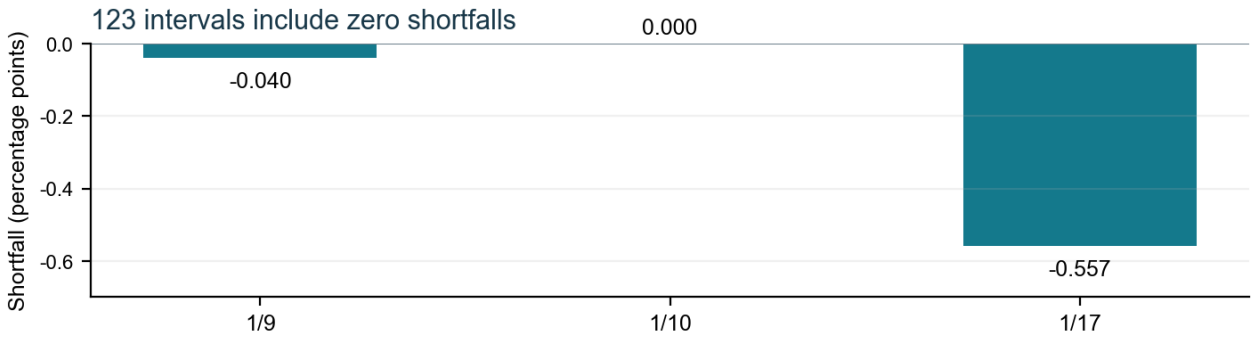
DD is downside deviation, not Issue 10's drawdown; r is a decimal daily net return. The denominator includes all 123 intervals. 52 are below zero; the rest contribute 0.

$$DD = \sqrt{\frac{\sum \min(r, 0)^2}{123}} \approx \sqrt{\frac{0.0047893027}{123}} \approx 0.00623999$$

Before running: How much does the positive 1/10 return contribute to squared shortfall?

```
shortfall = np.minimum(r, 0)
downside = np.sqrt(np.mean(shortfall**2))
dates = ["2024-01-09", "2024-01-10", "2024-01-17"]
pd.DataFrame({"Observation date": dates,
    "Net return (%)": (100*r.loc[dates]).round(5).values,
    "Shortfall (percentage points)": (100*shortfall.loc[dates]).round(5).values})
```

Observation date	Net return (%)	Shortfall (percentage points)
2024-01-09	-0.04	-0.04
2024-01-10	0.52714	0.0
2024-01-17	-0.55735	-0.55735



1/10 contributes 0; the 1/9 shortfall is -0.04 percentage points. Full-sample downside deviation is about 0.6240 percentage points, including both shortfall frequency and magnitude.

Discussion

How would the result change if the same squared-shortfall sum were divided by only 52 losing days?

Sources: [4].

ISSUE 09

Does a higher Sortino mean more account income?

Sortino replaces Sharpe's all-return volatility denominator with downside deviation. Both still evaluate the same account returns.

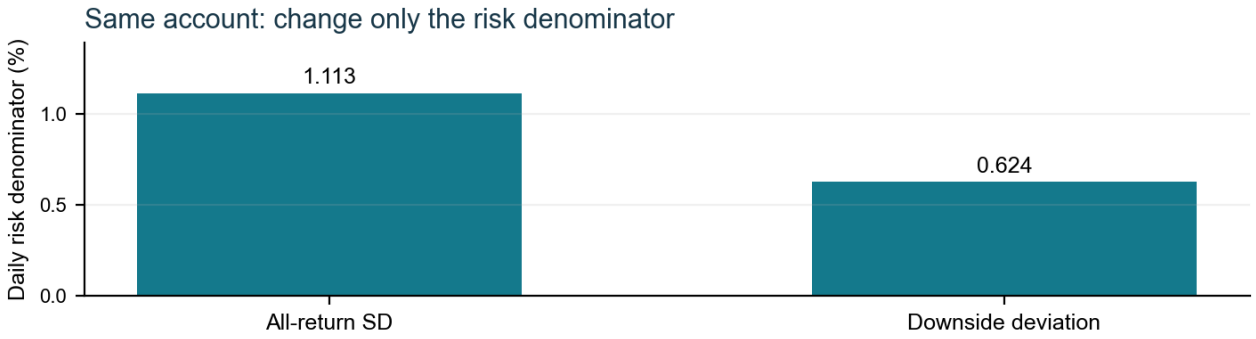
Keep the daily target at 0%. DD is Issue 8's downside deviation 0.6239986%, and the daily mean is 0.0905176%; both have the same frequency.

$$\text{Sortino}_{\text{annual}} = \frac{0.0905176}{0.6239986}\sqrt{252} \approx 2.302765$$

Before running: Does switching to downside deviation change terminal USD 1109.43?

```
sortino_daily = r.mean()/downside
sortino_annual = sortino_daily*np.sqrt(252)
pd.DataFrame({"Ratio": ["Sharpe", "Sortino"],
    "Conventional annualization": [sharpe_annual, sortino_annual]}).round(6)
```

Ratio	Conventional annualization
Sharpe	1.290932
Sortino	2.302765



The denominator falls from 1.1131% to 0.6240%, raising the ratio for a positive numerator. Trades and dollar income are unchanged.

√252 is a conventional scale, not an annual downside-risk identity. The ratio is undefined when DD is zero. With a negative numerator, a less negative ratio does not mean more income.

Discussion

Which daily variations enter the Sortino and Sharpe denominators? Explain this concrete difference to a classmate.

Sources: [2], [4].

ISSUE 10

How far did the account fall from a previous peak?

Drawdown compares current wealth with the highest value already reached, not yesterday's value or initial capital.

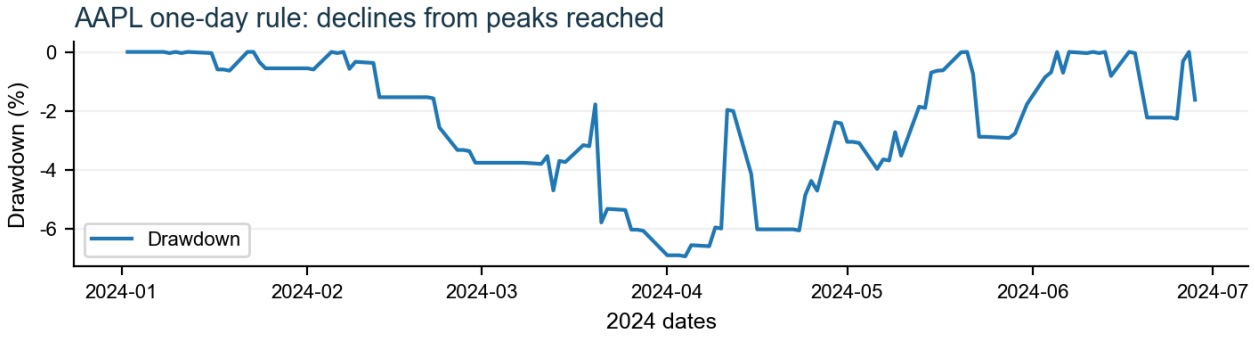
W is account wealth in USD, and M its peak through today, including initial 1000. D is drawdown, negative below the peak. Report maximum drawdown as a positive loss magnitude.

$$D = W/M - 1, \qquad 951.02510/1021.99903 - 1 \approx -6.944618\%$$

Before running: Does a positive terminal gain imply zero maximum drawdown?

```
peak = wealth.cummax().clip(lower=1000)
drawdown = wealth/peak-1
max_drawdown = -drawdown.min()
dates = ["2024-02-07", "2024-04-04", "2024-06-28"]
pd.DataFrame({"Observation date": dates,
    "Account (USD)": wealth.loc[dates].round(2).values,
    "Peak reached (USD)": peak.loc[dates].round(2).values,
    "Drawdown (%)": (100*drawdown.loc[dates]).round(4).values})
```

Observation date	Account (USD)	Peak reached (USD)	Drawdown (%)
2024-02-07	1022.0	1022.0	0.0
2024-04-04	951.03	1022.0	-6.9446
2024-06-28	1109.43	1127.76	-1.6254



From the 2/7 peak 1022.00 to the 4/4 trough 951.03, maximum decline is 6.9446%. Terminal gains do not erase this decline.

Discussion

Why is 4/4 wealth 951.02510 divided by initial 1000, giving -4.8975%, not maximum drawdown?

Sources: [10].

ISSUE 11

Why has a rebound not yet recovered the peak?

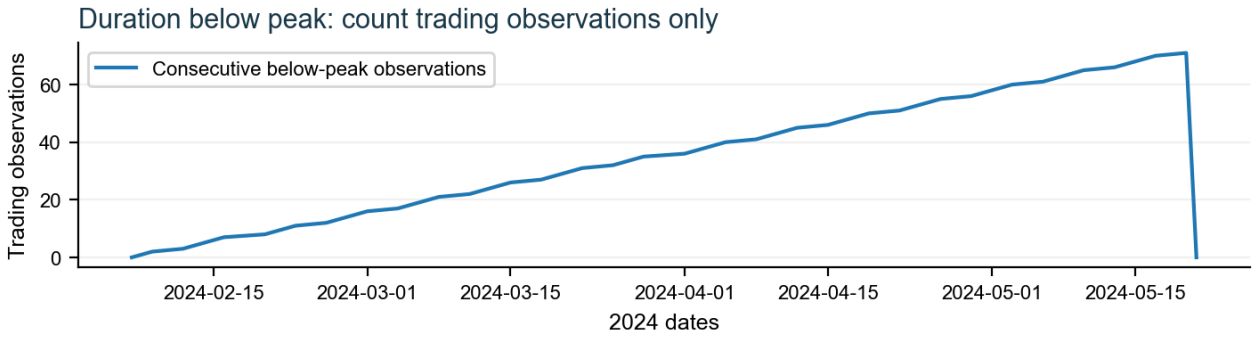
Separate drawdown depth from duration: one rising day need not recover the previous peak.

The 2/7 peak is USD 1021.99903. Count below-peak closes starting at the next observation, 2/8. Recovery occurs when wealth first meets or exceeds that peak; reset the count to zero then.

Before running: Wealth exceeds initial 1000 on 5/20; does it also exceed the 2/7 peak 1021.99903?

```
counts = []
count = 0
for depth in drawdown:
    count = count+1 if depth < -1e-12 else 0  # Allow floating-point error: matching the peak is not below it
    counts.append(count)
duration = pd.Series(counts, index=wealth.index)
dates = ["2024-04-04", "2024-05-20", "2024-05-21"]
pd.DataFrame({"Observation date": dates,
    "Account (USD)": wealth.loc[dates].round(5).values,
    "Below peak (observations)": duration.loc[dates].values})
```

Observation date	Account (USD)	Below peak (observations)
2024-04-04	951.0251	39
2024-05-20	1021.88139	71
2024-05-21	1028.88871	0



At the deepest point on 4/4, the below-peak count is 39. On 5/20, wealth is still about USD 0.1176 short, at count 71. The count resets only when 5/21 exceeds the old peak.

71 observations are not 71 calendar days. Count from the observation after the peak. If recovery has not occurred by sample end, report that it was not observed; do not treat the final date as recovery.

Discussion

With 5/20 wealth of USD 1021.88139, how much is needed to recover the 2/7 peak?

Sources: [5], [10].

ISSUE 12

Can an equal-percentage rise recover a 6.9446% decline?

Recovery starts from a smaller trough denominator, so the required percentage rise exceeds the original percentage decline.

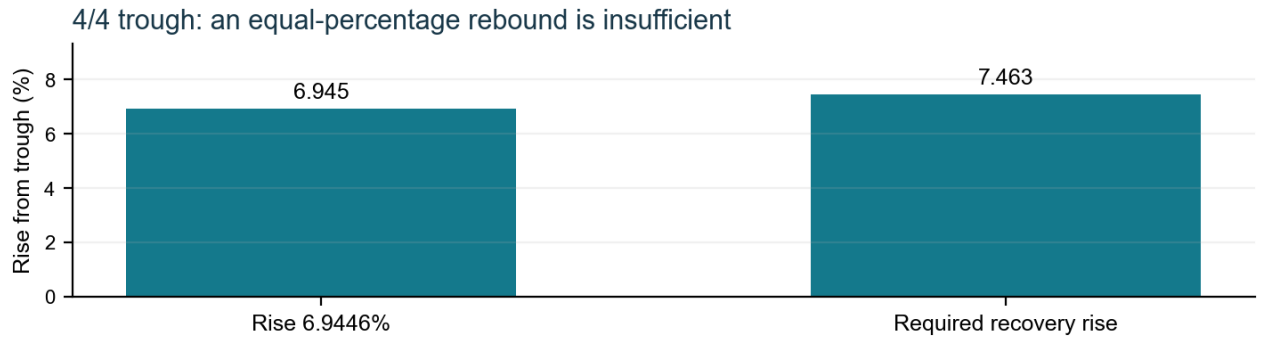
The trough is USD 951.02510 and old peak USD 1021.99903. Recover USD 70.97393; divide this amount by the trough, not the old peak.

$$\frac{1021.99903 - 951.02510}{951.02510} \approx 7.4629\%$$

Before running: Does multiplying the trough by 1.069446 recover USD 1022?

```
trough = drawdown.idxmin()
old_peak = peak[trough]
trough_value = wealth[trough]
required_gain = old_peak/trough_value-1
equal_percent_rebound = trough_value*(1+max_drawdown)
pd.DataFrame({"After the trough": ["Rise by original decline 6.9446%", "Recover old peak"],
    "Account (USD)": [equal_percent_rebound, old_peak]}).round(2)
```

After the trough	Account (USD)
Rise by original decline 6.9446%	1017.07
Recover old peak	1022.0



About 7.4629% is required to regain the peak, not a forecast of recovery time. This sample exceeds it only on 5/21.

Discussion

Why do the decline and recovery have equal dollar amounts but different percentages?

Sources: [10].

ISSUE 13

Does higher terminal wealth mean a smaller interim decline?

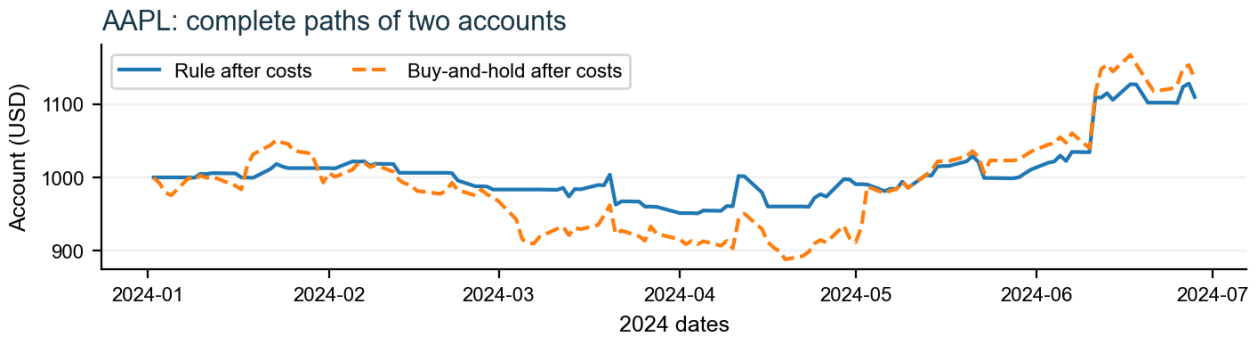
With the same dates and capital, compare terminal dollar wealth separately from declines below earlier peaks.

The gross and net rule accounts differ only in costs. Buy-and-hold also changes holding periods: enter on 1/2, pay 4bp and do not sell at the end. Cash earns no interest.

Before running: If buy-and-hold has higher terminal wealth, is its maximum drawdown also smaller?

```
gross_wealth = 1000*(1+gross).cumprod(skipna=False)
buy_hold = 1000*(1-.0004)*close/close.iloc[0]
cash = pd.Series(1000.0, index=close.index)
paths = {"Rule before costs": gross_wealth, "Rule after costs": wealth,
         "Buy-and-hold after costs": buy_hold, "Cash": cash}
comparison = []
for name, path in paths.items():
    depth = max(0., -(path/path.cummax().clip(lower=1000)-1).min())
    comparison.append([name, path.iloc[-1], 100*depth])
pd.DataFrame(comparison,
    columns=["Method", "Terminal (USD)", "Maximum drawdown (%)"]).round(3)
```

Method	Terminal (USD)	Maximum drawdown (%)
Rule before costs	1134.059	6.272
Rule after costs	1109.433	6.945
Buy-and-hold after costs	1134.108	15.463
Cash	1000.0	0.0



The rule ends at USD 1109.43 with drawdown 6.945%; buy-and-hold ends at USD 1134.11 with drawdown 15.463%. Terminal wealth and interim decline give different comparisons.

Buy-and-hold entry cost is already in the account and drawdown, not the later 123 market daily returns. Differences across strategies cannot all be attributed to costs.

Discussion

Which two table rows isolate the effect of 4bp costs?

Sources: [1], [5].

ISSUE 14

Does an overall gain imply a gain in every calendar subperiod?

Split the continuing account into months 1 to 3 and 4 to 6. Do not reinvest 1000 or enter again in month 4.

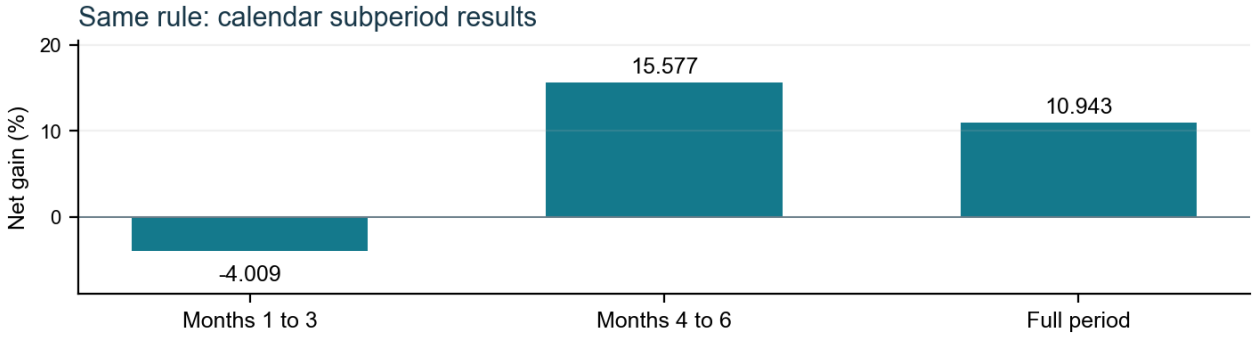
Before running: Can an earlier loss and later gain still yield an overall gain?

```
periods = {"Months 1 to 3": r.loc[:"2024-03-31"],
           "Months 4 to 6": r.loc["2024-04-01":]}
period_rows = []
for name, sample in periods.items():
    period_rows.append([name, len(sample), 100*((1+sample).prod()-1)])
pd.DataFrame(period_rows,
    columns=["Period", "Daily intervals", "Net gain (%)"]).round(4)
```

Period	Daily intervals	Net gain (%)
Months 1 to 3	60	-4.0094
Months 4 to 6	63	15.5772

The first period runs from USD 1,000 at the 1/2 close to the 3/28 close: 60 intervals, about -4.00936%, leaving USD 959.91. The next continues from that 3/28 wealth to 6/28: 63 intervals, about +15.57724%. 4/1 ends its first interval (3/28→4/1); do not reset wealth to 1,000. Multiply the two growth factors to obtain full-period 10.9433%.

$$(1 - 0.0400936)(1 + 0.1557724) - 1 \approx 10.9433\%$$



The full-period gain hides an earlier decline. These are explicit calendar subperiods, not identified market regimes.

Discussion

Would entering anew with 1000 in April still give this page's second-period result?

Sources: [5], [7].

ISSUE 15

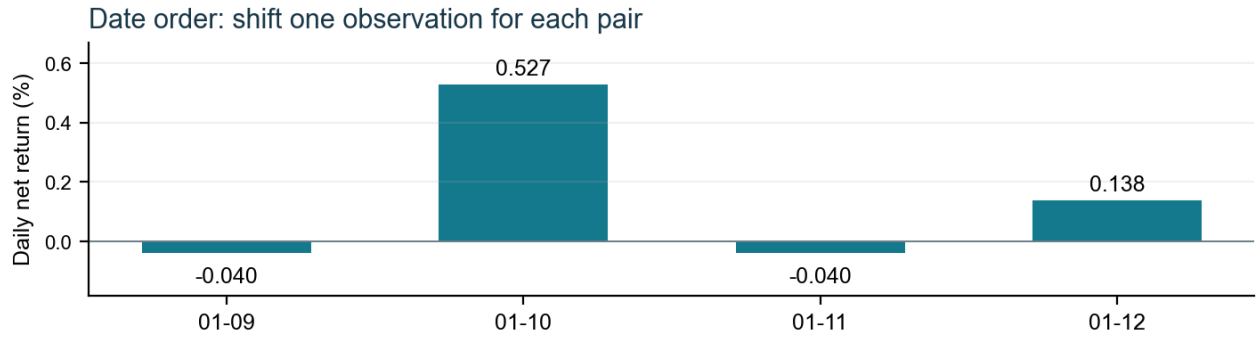
Why does day 10 appear twice in adjacent-return pairs?

To examine time dependence, pair each observation with the next: day 10 follows day 9 and precedes day 11.

Before running: Do 123 ordered observations paired with their successors give 123 or 122 pairs?

```
pairs = [("2024-01-09", "2024-01-10"),
         ("2024-01-10", "2024-01-11"),
         ("2024-01-11", "2024-01-12")]
pd.DataFrame([[a, b, 100*r[a], 100*r[b]] for a, b in pairs],
    columns=["First interval end", "Next interval end", "First return (%)", "Next return (%)"]).round(5)
```

First interval end	Next interval end	First return (%)	Next return (%)
2024-01-09	2024-01-10	-0.04	0.52714
2024-01-10	2024-01-11	0.52714	-0.04
2024-01-11	2024-01-12	-0.04	0.13781



9→10, 10→11 and 11→12 overlap. Day 10 is not treated as two independent market observations.

All 123 observations give 122 pairs. Independent daily resampling does not copy original neighbors together; block resampling retains their order within blocks.

Discussion

The next trading observation after 1/12 is 1/16. Does the weekend prevent a pair?

Sources: [3], [6].

ISSUE 16

How do same-direction daily deviations enter covariance?

Subtract each column's own mean, then multiply deviations. Same-direction deviations give positive products; opposite-direction deviations give negative products.

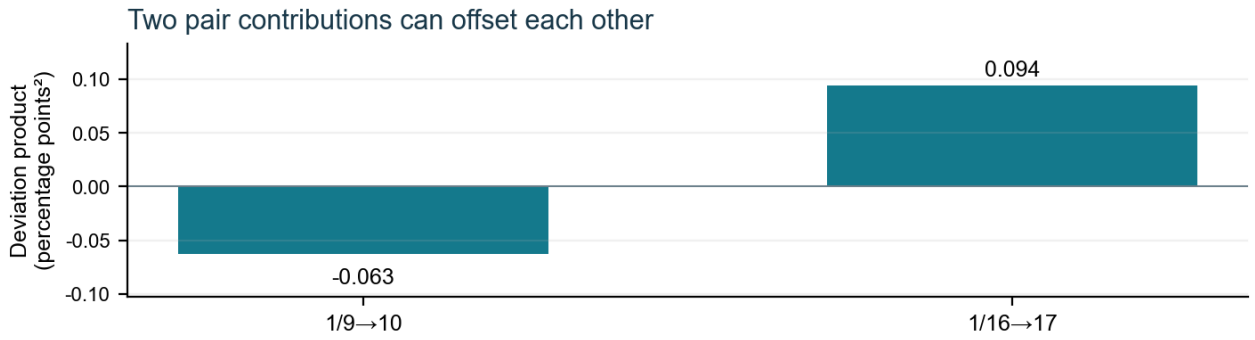
x is the first column and y the next column of net returns in percent. Across 122 pairs, their means are 0.1045826% and 0.0912595%. pp means percentage points; deviation products are in percentage points².

$$(-0.04 - 0.1045826)(0.5271400 - 0.0912595) \approx -0.063021 \text{ pp}^2$$

Before running: If 1/9 is below its column mean and 1/10 above its own, is their deviation product positive or negative?

```
previous = 100*r.iloc[:-1].to_numpy()
following = 100*r.iloc[1:].to_numpy()
products = (previous-previous.mean())*(following-following.mean())
covariance = products.sum()/(len(products)-1)
positions = [r.index.get_loc(d) for d in ["2024-01-09", "2024-01-16"]]
pd.DataFrame({"Pair endpoints": ["1/9→10", "1/16→17"],
    "Deviation product (percentage points²)": products[positions]}).round(6)
```

Pair endpoints	Deviation product (percentage points²)
1/9→10	-0.063021
1/16→17	0.093778



The first pair is negative and the second positive. Products across all 122 pairs sum to about -10.255770; divide by 121 for covariance -0.0847584 percentage points².

Discussion

Does one positive and one negative product in the figure imply zero covariance across all 122 pairs?

Sources: [2], [3].

ISSUE 17

How much does adding only two variances omit?

Squaring the sum of two column deviations gives their individual squares plus twice their product.

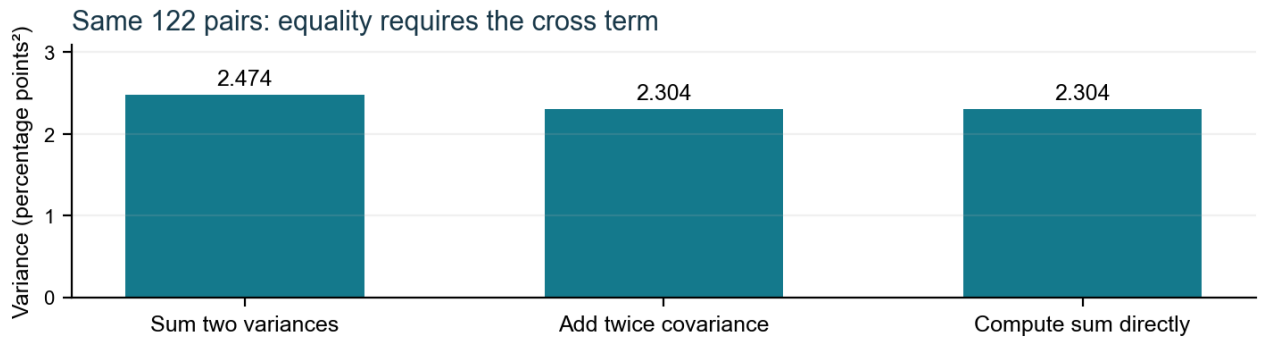
u and v are the previous issue's column deviations; s² denotes the corresponding sample variance and s_xy covariance. All terms use the same 122 pairs, denominator 121 and percentage points² units.

$$(u+v)^2 = u^2 + v^2 + 2uv \quad \Rightarrow \quad s^2_{x+y} = s^2_x + s^2_y + 2s_{xy}$$

Before running: Does adding only the two variances give the actual sample variance of the two-day sum?

```
pair_variance = np.var(previous+following, ddof=1)
zero_covariance = np.var(previous, ddof=1)+np.var(following, ddof=1)
lag_one = np.corrcoef(previous, following)[0, 1]
pd.DataFrame({"Calculation":
    ["Sum two variances only", "Include twice covariance", "Compute two-day sum directly"],
    "Variance (percentage points²)": [zero_covariance,
        zero_covariance+2*covariance, pair_variance]}).round(6)
```

Calculation	Variance (percentage points²)
Sum two variances only	2.473812
Include twice covariance	2.304296
Compute two-day sum directly	2.304296



2.473812 plus -0.169517 gives 2.304296. Sample adjacent correlation is about -0.068528, not proof of independence or future dependence.

Correlation divides covariance by the product of the two column SDs and has no units. Checking this adjacent correlation alone does not establish annualization conditions. This identity is also not the variance of a two-day compounded return.

Discussion

With negative covariance in the previous issue, does omitting it overestimate or underestimate this two-day sum's variance?

Sources: [2], [3], [11].

ISSUE 18

What does resampling a five-day block copy?

Circular block resampling copies 5 consecutive existing net returns. It neither downloads new data nor reruns trades.

77 is the random seed, reproducing the same sampling order here. It is neither block length nor sample size.

Before running: Is the date order within one block retained?

```
values = r.to_numpy()
rng = np.random.default_rng(77)
starts = rng.integers(0, len(values), int(np.ceil(len(values)/5)))
indices = (starts[:, None]+np.arange(5)) % len(values)
first_block = indices[0]
pd.DataFrame({"Original date": r.index[first_block],
    "Copied net return (%)": (100*values[first_block]).round(6)})
```

Original date	Copied net return (%)
2024-01-12	0.137812
2024-01-16	-0.04
2024-01-17	-0.557351
2024-01-18	0.0
2024-01-19	-0.04

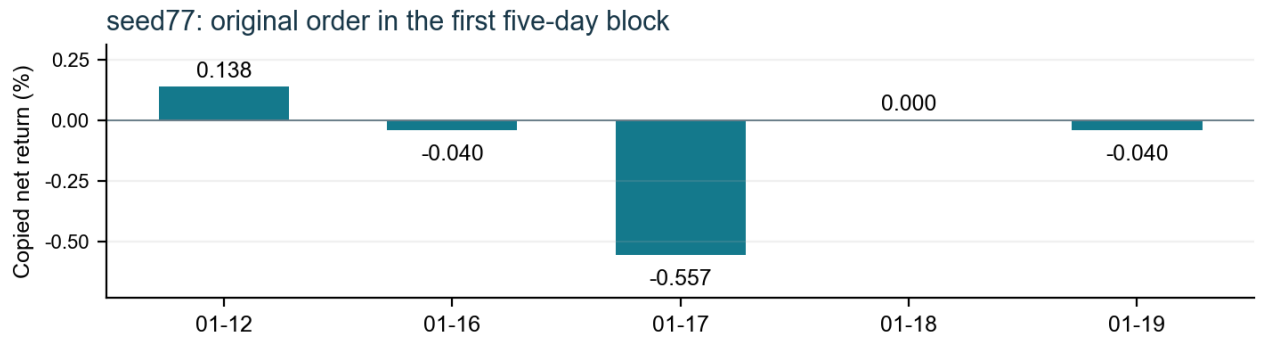


Figure and table show the same 5 original returns. Draw 25 blocks for 125 observations, then truncate to 123. Some original dates repeat; others are not drawn.

Adjacency is retained within blocks, not at their joins. Wrapping the end to the start is a circular-sampling setting, not an observed market transition.

Discussion

After joining blocks from different original dates, can this be called a newly observed 123-day backtest of the company?

Sources: [6], [9].

ISSUE 19

Why do three resamples give different Sharpe ratios?

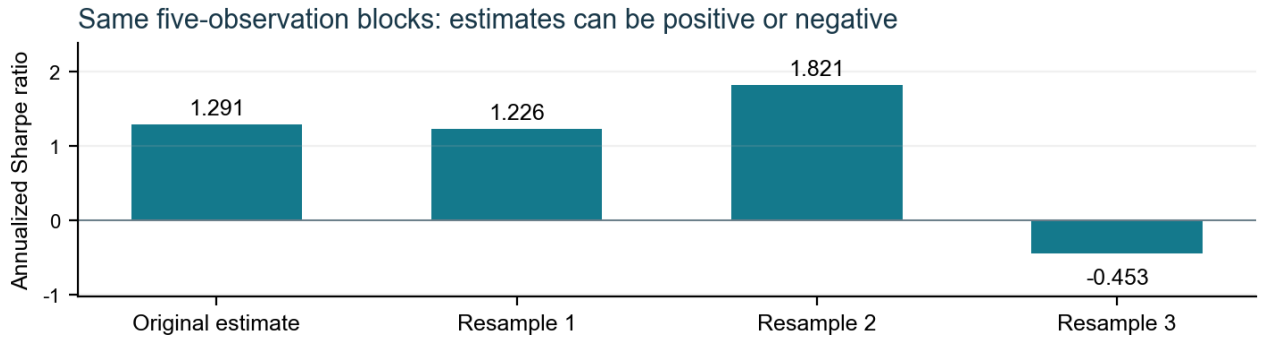
Different blocks change the mean and SD in each resample. Keep the zero daily reference, annualized scale and original data fixed.

Before running: Must all three resamples reproduce the original account's 1.290932?

```
rng = np.random.default_rng(77)
first_samples = []
for repetition in range(1, 4):
    starts = rng.integers(0, len(values), int(np.ceil(len(values)/5)))
    indices = (starts[:, None]+np.arange(5)) % len(values)
    sample = values[indices.ravel()[:len(values)]]
    mean, sd = sample.mean(), sample.std(ddof=1)
    ratio = mean/sd*np.sqrt(252) if sd > 0 else np.nan
    first_samples.append([repetition, 100*mean, 100*sd, ratio])
pd.DataFrame(first_samples, columns=["Resample count", "Mean (%)",
    "SD (%)", "Annualized Sharpe"]).round(6)
```

Resample count	Mean (%)	SD (%)	Annualized Sharpe
1	0.086305	1.117659	1.225819
2	0.136486	1.189744	1.821103
3	-0.024904	0.87333	-0.452682

$$\frac{0.0863048}{1.1176587}\sqrt{252} \approx 1.225819$$



1.225819, 1.821103 and -0.452682 are resampled estimates, not three realized annual investment results. Three resamples cannot summarize the distribution.

Discussion

Does a negative Sharpe in resample 3 imply a certain loss in the third future year?

Sources: [2], [6], [9].

ISSUE 20

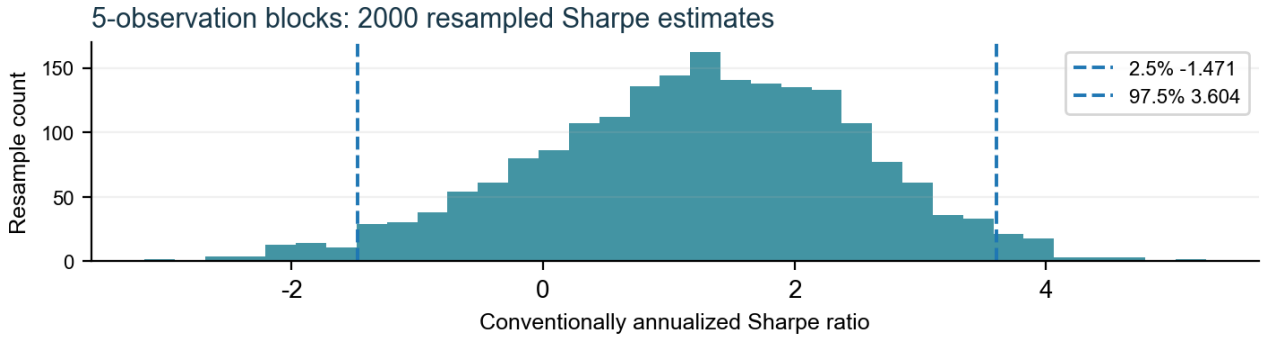
Must a positive Sharpe estimate have an entirely positive uncertainty interval?

Fix 5-observation blocks, seed77 and 2000 resamples. Sort Sharpe estimates and take the 2.5% and 97.5% quantiles to enclose the middle 95% of resampled estimates.

Before running: With original estimate 1.291 positive, must the resampled 2.5% quantile also be positive?

```
def bootstrap_sharpe(length):
    rng = np.random.default_rng(77)
    estimates = []
    for _ in range(2000):
        starts = rng.integers(0, len(values), int(np.ceil(len(values)/length)))
        indices = (starts[:, None]+np.arange(length)) % len(values)
        sample = values[indices.ravel()[:len(values)]]
        sd = sample.std(ddof=1)
        estimates.append(sample.mean()/sd*np.sqrt(252) if sd > 0 else np.nan)
    return np.array(estimates)
bootstrap = bootstrap_sharpe(5)
finite = bootstrap[np.isfinite(bootstrap)]
interval = np.quantile(finite, [.025, .5, .975])
pd.DataFrame([interval], columns=["2.5%", "Median", "97.5%"]).round(3)
```

2.5%	Median	97.5%
-1.471	1.277	3.604



[-1.471,3.604] includes zero; the median is 1.277, with 0 undefined estimates. A positive original estimate does not imply positive estimates for every data rearrangement.

This interval only rearranges the original 123 strategy net daily returns. It describes estimate variation under the specified resampling method, not a validated distribution for other market conditions or a 95% guarantee of future returns. It excludes rule-selection uncertainty, and the short sample does not guarantee coverage.

Discussion

What information is hidden by reporting 1.291 without the interval containing zero?

Sources: [3], [6], [8], [9].

ISSUE 21

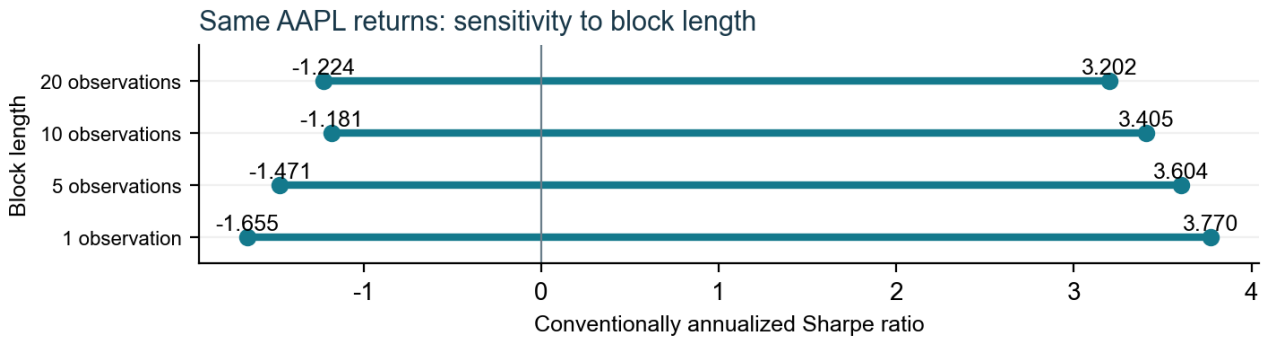
Must longer blocks give wider resampling intervals?

Change only block length. Keep returns, 2000 resamples, seed77, zero reference and the quantile method fixed.

Before running: Must retaining more adjacent order widen the estimate interval?

```
block_results = []
for length in [1, 5, 10, 20]:
    estimates = bootstrap_sharpe(length)
    valid = estimates[np.isfinite(estimates)]
    lower, upper = np.quantile(valid, [.025, .975])
    block_results.append([length, lower, upper, len(estimates)-len(valid)])
pd.DataFrame(block_results, columns=["Block (observations)", "2.5%", "97.5%",
    "Undefined count"]).round(3)
```

Block (observations)	2.5%	97.5%	Undefined count
1	-1.655	3.77	0
5	-1.471	3.604	0
10	-1.181	3.405	0
20	-1.224	3.202	0



The 1-observation interval is [-1.655,3.770], versus [-1.224,3.202] for 20 observations. The latter is narrower here; longer blocks do not necessarily give wider intervals.

Length 1 samples observations independently. Longer blocks retain more original neighbors but draw fewer blocks. The same seed does not mean different lengths draw the same blocks. This examines settings, not a universally correct length.

Discussion

The 20-observation interval is narrowest but its lower bound is -1.224. Does this establish Sharpe above zero?

Sources: [3], [6], [8], [9].

ISSUE 22

What is hidden when only the selected highest return is reported?

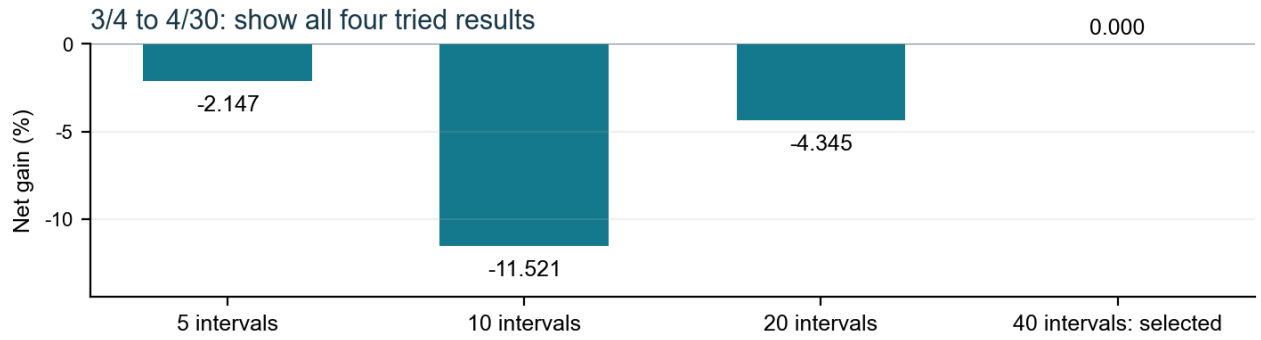
Also try rules requesting stock when Close exceeds its value 5, 10, 20 or 40 trading intervals earlier, otherwise cash. This demonstrates a search, without replacing the original one-day account.

All use next-close execution and 4bp per direction. Select over the 41 returns from 3/4 to 4/30. The 40-interval signal is computable on 2/29; the 3/4 position uses this known nonpositive signal, not a missing value filled with zero.

Before running: Does selecting the largest of four results, above their median, prove predictive ability?

```
candidates = {}
for lookback in [5, 10, 20, 40]:
    feature = close/close.shift(lookback)-1
    s = feature.gt(0).astype(float).where(feature.notna())
    h = s.shift(2).fillna(0)
    after_trade = s.shift(1).fillna(0)
    after_trade.iloc[-1] = h.iloc[-1]
    changes = (after_trade-h).abs()
    candidates[lookback] = asset_return.where(h.eq(1), 0)-.0004*changes
candidates = pd.DataFrame(candidates)
selection = candidates.loc["2024-03-04":"2024-04-30"]
selection_gain = (1+selection).prod()-1
chosen = selection_gain.idxmax()
pd.DataFrame({"Lookback intervals": selection_gain.index,
    "Selection-period net gain (%)": (100*selection_gain).round(3).values})
```

Lookback intervals	Selection-period net gain (%)
5	-2.147
10	-11.521
20	-4.345
40	0.0



Select 40 intervals at 0.000%: all 41 selection-period intervals hold zero-interest cash without trading, not a gain. Other results are -2.147%, -11.521% and -4.345%. Highest does not mean positive; this example does not estimate overfitting probability.

Discussion

What important fact would publishing only the 40-interval rule's 0% hide?

Sources: [7].

ISSUE 23

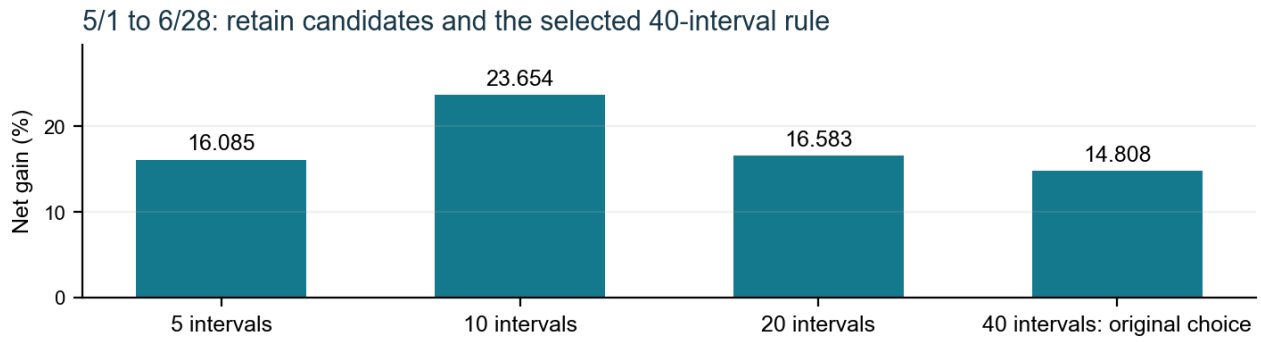
Does the previously selected rule remain highest later?

The 40-interval rule was selected by the end of month 4. Then examine 5/1 to 6/28; do not relabel the later winner as the original choice.

Before running: Does ranking highest in months 3 to 4 guarantee the same in months 5 to 6?

```
evaluation = candidates.loc["2024-05-01":"2024-06-28"]
evaluation_gain = (1+evaluation).prod()-1
pd.DataFrame({"Lookback intervals": selection_gain.index,
    "Originally selected": selection_gain.index == chosen,
    "Selection period (%)": (100*selection_gain).round(3).values,
    "Later period (%)": (100*evaluation_gain).round(3).values})
```

Lookback intervals	Originally selected	Selection period (%)	Later period (%)
5	False	-2.147	16.085
10	False	-11.521	23.654
20	False	-4.345	16.583
40	True	0.0	14.808



The originally selected 40-interval rule later returns 14.808%; the 10-interval rule is higher at 23.654%. Reselecting 10 using that later period turns it into selection data too.

Candidates operate continuously; only dates divide the periods. Do not reinvest 1000 and switch strategy on 4/30. Each period has 41 returns, with slightly different calendar spans. This is a retrospective example using known history, not an untouched live test. The selected 40-interval rule not ranking first here does not prove such selection must fail.

Discussion

If you first deployed USD 1000 in the 40-interval rule on 4/30, what else would need calculating?

Sources: [7].

ISSUE 24

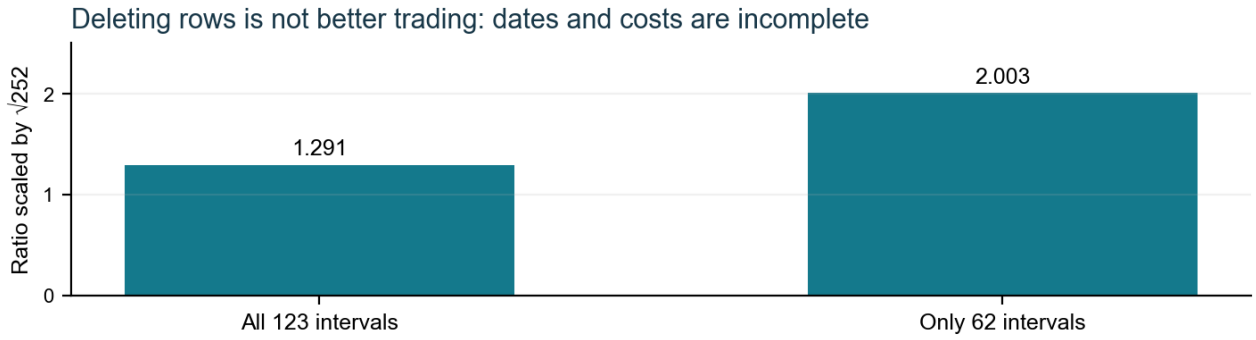
Why does deleting cash days improve the reported numbers?

Return to the one-day rule. Compare all 123 intervals with stock-held days only; the latter is not another executable strategy.

Before running: Does deleting intervals held entirely in cash before purchase also delete purchase costs?

```
stock_only = r[held.iloc[1:].eq(1)]
subset_sharpe = stock_only.mean()/stock_only.std(ddof=1)*np.sqrt(252)
subset_gain = (1+stock_only).prod()-1
pd.DataFrame({"Dates retained": ["All account days", "Stock-held days only"],
    "Interval count": [len(r), len(stock_only)],
    "Compounded gain (%)": [100*observed_gain, 100*subset_gain],
    "Sharpe ×√252": [sharpe_annual, subset_sharpe]}).round(4)
```

Dates retained	Interval count	Compounded gain (%)	Sharpe ×√252
All account days	123	10.9433	1.2909
Stock-held days only	62	12.1931	2.0028



Larger figure values come from deleted rows. Waiting dates and purchase costs are missing, and 62 intervals do not cover the full continuous calendar. They do not establish an advantage on the same annual scale.

Complete-account net gain is 10.9433%; deleting rows raises it to 12.1931%. The full period has 28 purchases and 27 sales; purchase intervals still hold cash. Deleted-row results cannot replace the actual USD 1109.43 account.

Discussion

How many of the 55 trading costs remain if only 62 stock-held intervals are retained?

Sources: [1], [2], [3], [5].

References

Price sources and reproduction conditions

[1] yfinance: historical prices

[2] Sharpe (1994): the Sharpe ratio

[3] Lo (2002): The Statistics of Sharpe Ratios

[4] CFA Institute: Sortino and downside deviation

[5] GIPS: return and risk presentation

[6] Politis and Romano: circular block resampling

[7] White（2000）：Data Snooping

[8] NumPy: quantiles

[9] arch: time-series resampling

[10] MSCI: maximum drawdown (page 51)

[11] NumPy: correlation coefficients

AAPL daily data: 2024/1/2 to 6/28, 124 observations. Reuse the Yahoo/yfinance snapshot retained on 2026/9/14. Trading rules, executions and costs are course assumptions, not actual trade records.

Display prices to 2 decimal places while retaining source precision in calculations. Reproduce using this version's recorded data; do not download replacements for its results.

Python 3.12.14；numpy 2.2.6；pandas 2.2.3；matplotlib 3.10.8；yfinance 1.2.0。